

1. Experimental Evaluation

This section describes the experimental methodology used to evaluate the impact of proactive IC/PIC generation. The goal of the evaluation is to compare the proposed mechanism against the current behavior of the VM and against other execution scenarios that isolate different parts of the dispatch and compilation process.

The experiments are designed around synthetic benchmarks that exercise message sends with a controlled degree of polymorphism. This makes it possible to vary independently the number of send sites, the number of receiver classes covered by the generated cache, and the number of iterations executed by each benchmark. The results of these experiments are presented in Section 2.

1.1. Research Questions

The evaluation is organized around the following research questions:

RQ1. Does proactive IC/PIC generation reduce execution time with respect to the current behavior of the VM?

This question evaluates the main hypothesis of the paper: if receiver type information is already available, generating the dispatch cache before execution should reduce the initial cost paid by dynamic message sends.

RQ2. How does the benefit of proactive cache generation change as the number of send sites increases?

This question studies whether the effect of the proposed mechanism scales with the number of dynamic sends performed by a method. Since each send site may require its own cache to become linked, methods with more send sites expose more opportunities for proactive generation.

RQ3. How does the benefit change as the number of benchmark iterations increases?

This question studies the interaction between proactive cache generation and warm-up. Reactive ICs and PICs become effective after the relevant send sites have executed and the VM has linked or extended their caches. Therefore, the relative benefit of proactive generation is expected to be larger in short executions and smaller when the benchmark runs for enough iterations to amortize the reactive construction cost.

RQ4. What is the effect of proactive cache generation on VM execution and code-space usage?

This question complements the execution-time analysis with VM-level metrics. In particular, it considers allocation-related statistics, JIT-compiled methods, generated closed PICs, and the amount of code space occupied by these structures.

1.2. Benchmarks

The main benchmarks are organized into families of methods. Each family is associated with a single message selector:

- **s***: repeated sends of `size`;
- **c***: repeated sends of `copy`;
- **a***: repeated sends of `asArray`.

Within each family, the numeric suffix indicates the number of send sites contained in the method. For example, `s1` contains one send of `size`, `s8` contains eight independent sends of `size`, and `s34` contains thirty-four independent sends. The same naming convention is used for the `copy` and `asArray` families.

The evaluated send-site counts are:

1, 2, 3, 5, 8, 13, 21, 34.

This produces twenty-four benchmark methods for the three main families. The receiver used by these methods is configured with up to six different classes:

OrderedCollection, Array, String, Heap, Interval, Set.

This configuration allows the experiments to cover monomorphic, bimorphic, and closed polymorphic cases with up to six receiver classes. The upper bound of six classes matches the scope of the current implementation, which generates closed PICs for a bounded number of entries.

In addition to the three selector-based families, the evaluation also includes a depth family. This benchmark is used to exercise a more dynamic dispatch pattern, where the receiver involved in the send is selected pseudo-randomly during the execution using a fixed seed. It complements the selector-based benchmarks by stressing the behavior of the dispatch cache when the receiver flow is less direct but still controlled by the experiment.

1.3. Execution Scenarios

Each benchmark is executed under four scenarios. These scenarios are designed to separate the effect of normal VM behavior, explicit JIT compilation, worst-case dispatch behavior, and proactive cache generation.

Scenario	Description
CB	Current behavior of the VM, without proactive cache generation
WC	Worst-case execution, where cache stabilization is intentionally disrupted
JIT	Methods are explicitly compiled by the JIT before measurement
PIC	The proposed scenario, with ICs/PICs generated proactively

Table 1

Execution scenarios used in the experimental evaluation.

The **Current Behaviour** scenario, abbreviated as **CB**, represents the standard execution of the system. In this scenario, the VM behaves normally: send sites are initially resolved through the usual dynamic dispatch path, and ICs or PICs are generated reactively as execution reaches those sites.

The **Worst Case** scenario, abbreviated as **WC**, represents an unfavorable execution mode for dynamic dispatch. In this scenario, cache stabilization is intentionally disrupted so that the benchmark does not benefit from the usual progressive linking of send sites. This scenario is not intended to represent normal use, but to provide a lower bound that highlights the cost of repeatedly paying dispatch overhead.

The **JIT** scenario isolates the effect of compiling the method to native code. In this case, the relevant methods are compiled by the JIT before measurement, but the dispatch caches are not generated proactively from LiveTyping information. This scenario is useful for distinguishing the benefit of native-code compilation from the additional benefit of installing ICs or PICs ahead of execution.

The **PIC** scenario corresponds to the mechanism proposed in this paper. Before executing the benchmark, the image uses the available LiveTyping information to request the generation of ICs or closed PICs for the selected send sites. The number of entries generated in the cache is controlled experimentally, from one to six entries.

These scenarios allow the evaluation to compare proactive generation against both the normal adaptive behavior of the VM and a JIT-only baseline. They also make it possible to analyze whether the observed benefit comes from avoiding reactive cache construction rather than from JIT compilation alone.

1.4. Experimental Setup

The experiments were executed in a controlled environment in order to improve reproducibility and reduce external interference. Each run was performed inside an isolated container on a Proxmox-based environment. Each container was assigned two virtual CPUs and 1GB of RAM.

The experiments were executed using a Cuis image [1, 2] prepared in *reader* mode. This image runs without a graphical user interface and receives commands through standard input. This setup allows

the benchmarks to be launched automatically from shell scripts, without manual interaction and with the same initialization path across runs.

The execution was orchestrated by scripts that enumerate the experimental configurations and invoke the corresponding Smalltalk entry points. The parameters varied by the scripts include:

- the execution scenario: CB, WC, JIT, or PIC;
- the benchmark family: size, copy, asArray, or depth;
- the method variant, determined by the number of send sites;
- the number of entries generated in the PIC, from one to six;
- the number of benchmark iterations.

For each configuration, the benchmark is executed repeatedly to reduce the effect of measurement noise. Each run starts from a fresh image, so that caches generated by previous executions do not affect later measurements. This is important because the effect being measured is precisely the behavior of dispatch caches during the beginning of execution.

The benchmark iteration parameter controls how many effective executions of the method under evaluation are included in one measurement. The values used in the experiments are:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 50, 100.

Before each measurement, garbage collection is explicitly triggered to reduce carry-over effects from allocations performed in previous runs and to start each measurement from a more comparable heap state. During the timed execution, a measurement is only accepted if no garbage collection occurs. If a garbage collection is detected during the measured block, that measurement is discarded and retried. Execution time is recorded in microseconds.

The VM was executed with display, sound, and event handling disabled:

```
-vm-display-null -vm-sound-null -noevents
```

This configuration reduces unrelated VM activity during the experiments and makes the measurements more focused on the execution of the benchmark itself.

1.5. Metrics

The primary metric is execution time, measured in microseconds. For each benchmark configuration, repeated measurements are collected and first filtered to remove outliers using the interquartile range (IQR) criterion. The remaining measurements are then summarized using the median. The median is used instead of the mean to further reduce the influence of residual system noise and other non-deterministic effects of the execution environment.

The execution-time results are analyzed in three complementary ways. First, the scenarios are compared directly, especially PIC against CB and JIT. Second, the number of send sites is varied while keeping the number of benchmark iterations fixed. Third, the number of benchmark iterations is varied while keeping the number of send sites fixed. This organization is reflected in the structure of the results section.

In addition to absolute execution time, the evaluation also reports the relative gain of the PIC scenario with respect to the current behavior of the VM. This value is computed as:

$$\text{Gain}_{PIC/CB} = \frac{T_{CB} - T_{PIC}}{T_{CB}} \times 100.$$

Here, T_{CB} is the execution time under the current behavior of the VM, and T_{PIC} is the execution time when proactive cache generation is used. Positive values indicate that proactive generation improves execution time with respect to the current behavior.

The evaluation also records VM-level statistics before and after each run. These statistics are used to complement the execution-time results and to understand the runtime effects of the proposed mechanism. In particular, the results report allocation-related statistics and JIT/PIC code-space usage.

The VM-level statistics reported in the results section are:

- used bytes in new space;
- number of methods compiled by the JIT;
- space occupied by JIT-compiled methods;
- number of generated closed PICs;
- space occupied by generated closed PICs.

The JIT and PIC code-space metrics are obtained through VM-level instrumentation that traverses the method zone and reports both the number of generated structures and the amount of space they occupy. These metrics are important because proactive generation can improve execution time while also increasing the amount of code generated before or during benchmark execution.

Together, these metrics allow the evaluation to distinguish three effects: the direct execution-time impact of proactive cache generation, the way this impact scales with send sites and iterations, and the code-space cost associated with generating dispatch caches ahead of execution.

2. Results

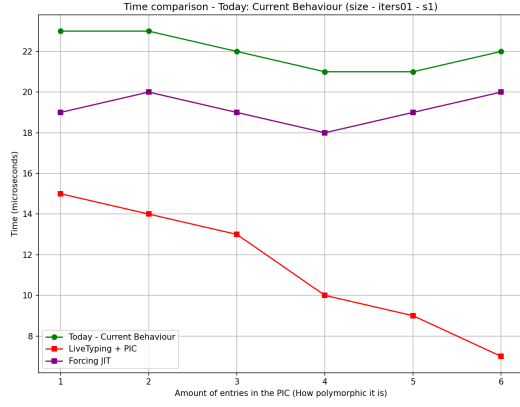
This section reports absolute execution time and relative gain. Lower absolute time means faster execution. Positive values of $\text{Gain}_{\text{PIC/CB}}$ indicate that proactive PIC generation improves over the current baseline.

2.1. Overall Execution Time

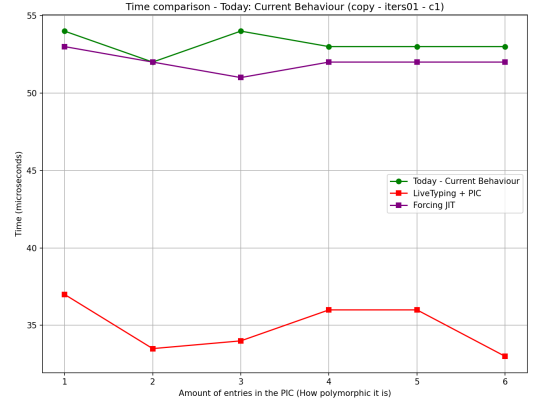
We first compare the current baseline, explicit JIT compilation, and proactive PIC generation across the four benchmark families: `size`, `copy`, `asArray`, and `depth`. The comparison focuses on the initial configuration, with one benchmark iteration and one send site, where the cost paid before reactive cache generation stabilizes the send site is most visible.

Figure 1 shows that the PIC scenario obtains the best or near-best execution time in this initial configuration. The clearest improvements appear with respect to the current baseline, especially when the initial dispatch cost is a large fraction of the total execution time. The comparison with JIT indicates that native-code compilation alone does not remove this initial cost: proactively installed dispatch caches still help during early execution. The effect is more pronounced in dispatch-dominated benchmarks such as `size` and `depth`, and smaller in `copy` and `asArray`, where allocation and heap activity also contribute to the measured time.

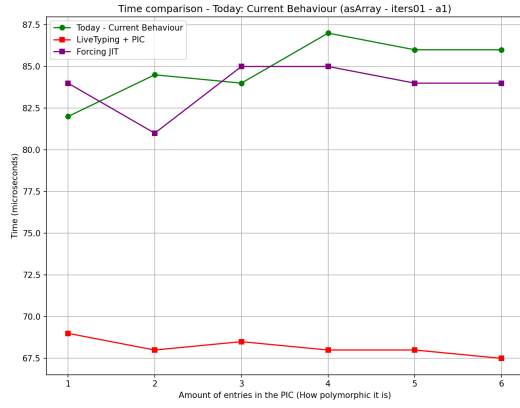
The WC scenario is included as a diagnostic reference for an unfavorable dispatch-cache configuration. In the measurements, it behaves as expected and does not provide the most competitive execution times. Since the goal of the evaluation is to compare proactive cache generation against the current VM behavior and against explicit JIT compilation, the following subsections focus mainly on CB, JIT, and PIC.



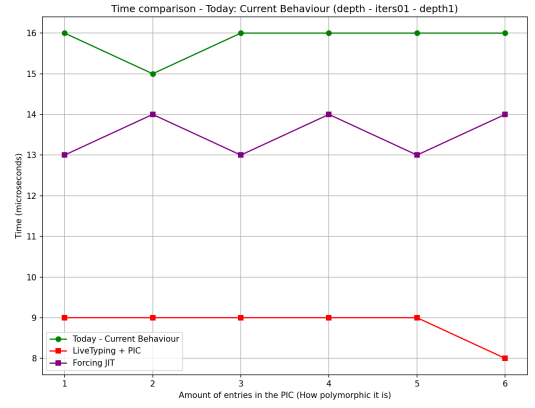
(a) size



(b) copy



(c) asArray



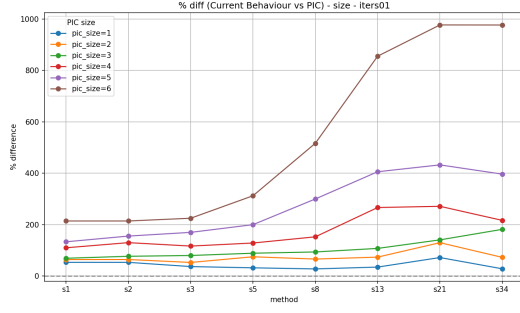
(d) depth

Figure 1: Execution-time comparison among the current baseline, the JIT scenario, and proactive PIC generation in the initial configuration (1 benchmark iteration, 1 send site). Each subplot corresponds to one benchmark family.

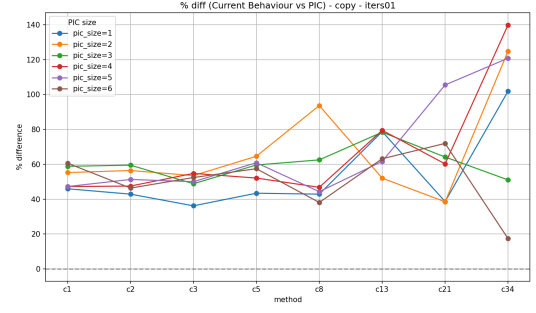
2.2. Impact of the Number of Send Sites

We next fix the number of benchmark iterations to one and vary the number of send sites. This setting emphasizes the beginning of execution, where each send site has little opportunity to amortize the cost of reactive cache construction. As a result, the experiment highlights whether generating ICs or PICs in advance becomes more beneficial as the number of optimized send sites increases.

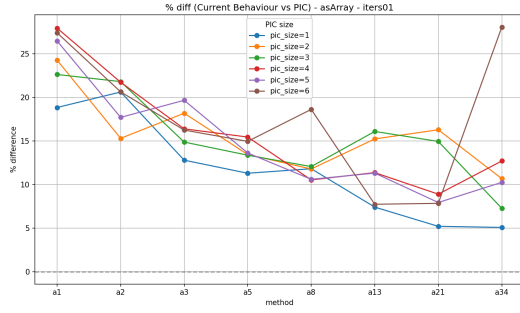
Figure 2 shows how execution time changes as more send sites are added to the benchmark. If the PIC scenario separates from the baseline as the number of sends increases, this indicates that proactive cache generation is avoiding repeated warm-up costs across multiple sites. If the curves converge instead, the result suggests that reactive cache construction is not the dominant cost in that configuration, or that the overhead introduced by the proactive PICs offsets part of the expected benefit.



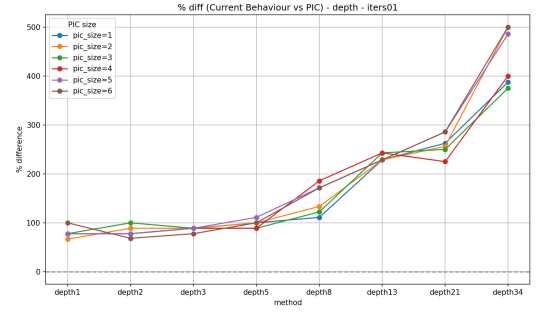
(a) size



(b) copy



(c) asArray



(d) depth

Figure 2: Impact of the number of message send sites on the relative gain of proactive PIC generation. Each subplot reports $\text{Gain}_{\text{PIC}/\text{CB}}$ while keeping the number of benchmark iterations fixed to one. Higher positive values indicate a larger improvement with respect to the current baseline.

2.3. Impact of the Number of Iterations

We now evaluate the complementary dimension: the number of send sites is fixed to thirty-four while the number of benchmark iterations increases. This experiment measures how the relative gain evolves as the same send sites are executed repeatedly.

Reactive ICs and PICs become effective only after the first executions of a send site. Therefore, Figure 3 compares short executions, where reactive caches have had fewer opportunities to be constructed, with longer executions, where that cost can be amortized. When the curves approach zero, the relative difference between PIC and the current baseline becomes smaller because the baseline has progressively constructed and reused its own caches.

Table 2 summarizes the percentage gain of the PIC scenario with respect to the current baseline for representative configurations. Since each configuration was evaluated with PICs from one to six entries, each cell reports the range of gains observed across those PIC sizes.

The table makes the main trend explicit: the largest gains appear in short executions and in methods with many send sites, while gains tend to decrease when the baseline has enough iterations to amortize reactive cache construction. The effect is weakest in asArray, where other costs dominate more quickly, and strongest in depth, where the dynamic receiver pattern keeps dispatch behavior central to the measurement.

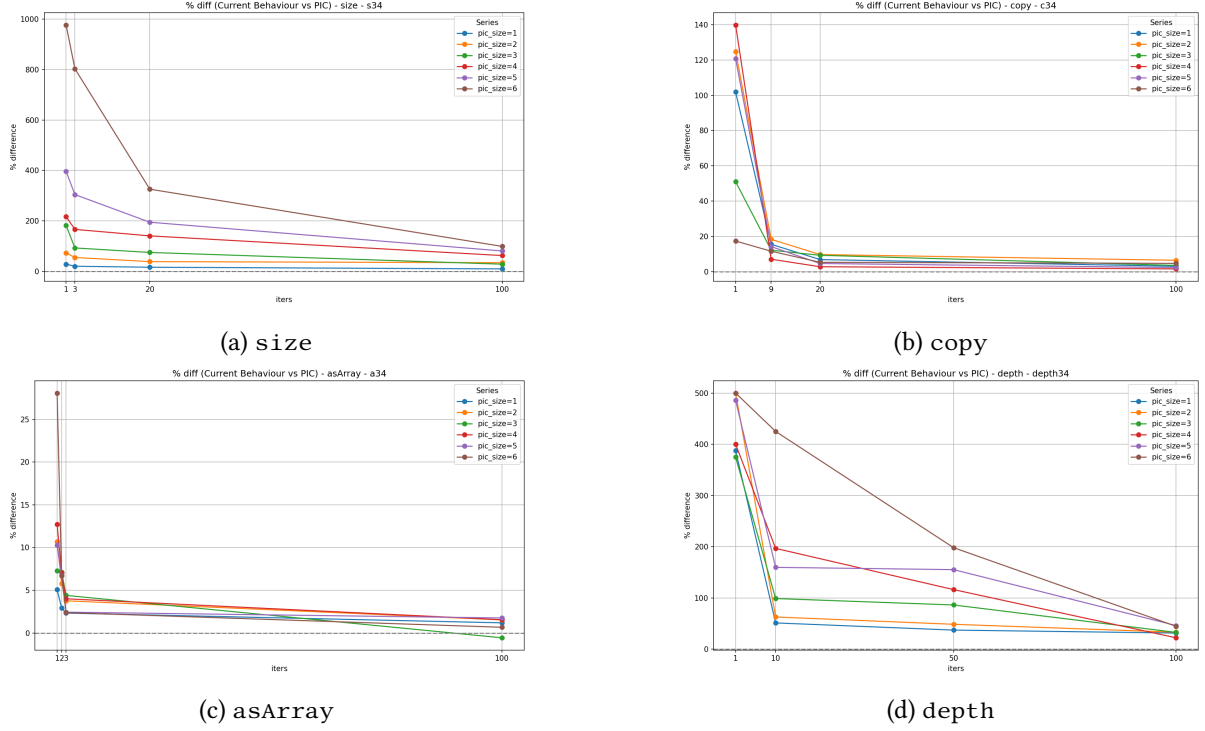


Figure 3: Impact of the number of benchmark iterations on the relative gain of proactive PIC generation. Each subplot reports $\text{Gain}_{\text{PIC}/\text{CB}}$ while keeping the number of send sites fixed to thirty-four. Higher positive values indicate a larger improvement with respect to the current baseline.

Configuration	Gain
1 iter., 1 send	54 — 214%
100 iter., 1 send	-2 — 23%
1 iter., 34 sends	31 — 972%
100 iter., 34 sends	10 — 98%

(a) size

Configuration	Gain
1 iter., 1 send	19 — 28%
100 iter., 1 send	-1 — 3%
1 iter., 34 sends	5 — 28%
100 iter., 34 sends	-1 — 2%

(c) asArray

Configuration	Gain
1 iter., 1 send	46 — 60%
100 iter., 1 send	-2 — 24%
1 iter., 34 sends	18 — 139%
100 iter., 34 sends	2 — 6%

(b) copy

Configuration	Gain
1 iter., 1 send	67 — 100%
100 iter., 1 send	14 — 50%
1 iter., 34 sends	375 — 499%
100 iter., 34 sends	22 — 46%

(d) depth

Table 2

Range of relative gains of proactive PIC generation with respect to the current baseline for representative configurations. Each subtable corresponds to one benchmark family. Values report the minimum and maximum of $\frac{T_{\text{CB}} - T_{\text{PIC}}}{T_{\text{CB}}} \times 100$ observed when varying the number of PIC entries from one to six. Positive values indicate that proactive PIC generation is faster than the current baseline.

2.4. VM Statistics and Code-space Usage

Finally, we complement the timing results with VM-level statistics. Proactive generation reduces part of the initial dispatch cost by materializing dispatch structures earlier; this introduces additional VM-level work in new space, JIT-compiled methods, generated closed PICs, and code-space usage.

Table 3 summarizes representative configurations with one and thirty-four send sites, and with one and one hundred benchmark iterations.

The table shows the cost of installing dispatch caches proactively. The PIC scenario generally increases the number of generated code structures and the code space occupied by JIT-compiled methods and

Scenario	Configuration	Δ new space	Δ #JIT	Δ JIT bytes	Δ #PICs	Δ PIC bytes
CB	1 iter., 1 send	1.98	10	4.72	2	0.42
PIC	1 iter., 1 send	10.44	58	28.95	15	3.16
CB	1 iter., 34 sends	1.98	10	5.43	35	7.38
PIC	1 iter., 34 sends	331.82	58	29.66	48	10.12
CB	100 iter., 1 send	173.69	10	4.72	2	0.42
PIC	100 iter., 1 send	182.14	58	28.95	15	3.16
CB	100 iter., 34 sends	173.69	10	5.43	35	7.38
PIC	100 iter., 34 sends	503.52	58	29.66	48	10.12

(a) size

Scenario	Configuration	Δ new space	Δ #JIT	Δ JIT bytes	Δ #PICs	Δ PIC bytes
CB	1 iter., 1 send	8.33	11	5.08	7	1.48
PIC	1 iter., 1 send	18.26	58	29.11	21	4.43
CB	1 iter., 34 sends	170.95	11	5.79	40	8.44
PIC	1 iter., 34 sends	525.99	58	29.82	54	11.39
CB	100 iter., 1 send	667.19	11	5.08	7	1.48
PIC	100 iter., 1 send	675.86	58	29.11	21	4.43
CB	100 iter., 34 sends	11491.60	12	6.15	47	9.91
PIC	100 iter., 34 sends	11481.54	59	30.18	67	14.13

(c) asArray

Scenario	Configuration	Δ new space	Δ #JIT	Δ JIT bytes	Δ #PICs	Δ PIC bytes
CB	1 iter., 1 send	7.01	13	5.36	4	0.84
PIC	1 iter., 1 send	18.56	60	29.39	18	3.80
CB	1 iter., 34 sends	161.82	13	6.07	37	7.80
PIC	1 iter., 34 sends	560.79	60	30.10	51	10.76
CB	100 iter., 1 send	643.15	13	5.36	4	0.84
PIC	100 iter., 1 send	652.91	60	29.39	18	3.80
CB	100 iter., 34 sends	11493.29	14	6.43	41	8.65
PIC	100 iter., 34 sends	11490.30	61	30.46	61	12.87

(b) copy

Scenario	Configuration	Δ new space	Δ #JIT	Δ JIT bytes	Δ #PICs	Δ PIC bytes
CB	1 iter., 1 send	0.93	30	9.62	0	0.00
PIC	1 iter., 1 send	6.00	80	33.27	12	2.53
CB	1 iter., 34 sends	0.93	30	10.30	0	0.00
PIC	1 iter., 34 sends	249.93	80	33.95	45	9.49
CB	100 iter., 1 send	64.15	35	10.07	2	0.42
PIC	100 iter., 1 send	68.86	80	33.27	13	2.74
CB	100 iter., 34 sends	64.15	35	10.75	35	7.38
PIC	100 iter., 34 sends	310.09	80	33.95	46	9.70

(d) depth

Table 3

VM-level statistics for representative configurations with one and thirty-four send sites, and with one and one hundred benchmark iterations. Each subtable compares the current baseline with proactive PIC generation for one benchmark family. Space values are reported in KB.

closed PICs. It also shows that copy and asArray have larger new-space growth than size and depth, especially in longer executions, indicating that object allocation and heap usage increasingly affect the measured time. These measurements make the main trade-off explicit: proactive generation moves dispatch-cache construction earlier, reducing warm-up cost at the price of additional VM-level work and code-space usage.

3. Discussion

The results show that proactive IC/PIC generation can improve the initial execution of dynamic message sends by moving part of the cache construction work before the send sites are executed. This section discusses the implications of these results, the trade-offs introduced by the approach, and the main limitations of the current implementation.

The discussion is organized around the research questions introduced in Section 1.1. The benefits discussed first relate mainly to RQ1, RQ2, and RQ3: whether proactive IC/PIC generation improves execution time, how this effect changes with the number of send sites, and how it evolves as the number of iterations increases. The trade-offs then address RQ4, focusing on the VM-level cost of materializing dispatch caches proactively. Finally, the limitations describe the scope in which these observations should be interpreted.

3.1. Benefits

With respect to RQ1, the main benefit of the proposed approach is that it reduces the cost paid at the beginning of execution. In the current VM behavior, a send site must be executed before the VM can observe the receiver class, perform method lookup, and link or extend the corresponding cache. When LiveTyping information is already available, the image can use that information to request the generation of an IC or PIC before the first execution of the selected send site.

This effect is especially relevant for short executions. In those cases, the initial cost of reactive cache construction represents a larger fraction of the total execution time, because there are fewer later executions over which that cost can be amortized. Proactive generation addresses this situation by installing dispatch caches earlier, so that execution can start closer to the optimized state that the reactive mechanism would otherwise reach only after observing the relevant sends.

RQ2 considers how this effect changes as the number of send sites increases. Methods with more send sites expose more opportunities for the proposed mechanism, since each send site may otherwise

need to pay its own initial cache-construction cost. In this setting, proactive generation can reduce repeated warm-up costs across multiple dispatch sites in the same method.

RQ3 considers the effect of increasing the number of benchmark iterations. The results indicate that proactive generation is most directly connected to the warm-up phase of execution. As the number of iterations increases, the current baseline has more opportunities to construct and reuse its caches reactively. Therefore, the relative difference between proactive generation and the current behavior may become smaller as the initial cost is amortized over a longer execution.

Another benefit is that the approach gives an operational role to type information collected at the image level. LiveTyping was originally designed to record dynamic type information and make it available to development tools. In this work, that information is also used as an input to a VM-level optimization. This changes the direction of the usual type-feedback flow: instead of extracting type information from ICs and PICs, the image uses previously collected type information to guide the construction of ICs and PICs.

The approach also preserves the adaptive behavior of the VM. It does not require replacing the existing reactive mechanism, nor does it assume that the generated cache is complete forever. If execution later reaches a receiver class that is not represented in the proactively generated cache, the VM can still fall back to its normal lookup and cache update mechanisms. In this sense, proactive generation complements the existing dispatch optimization strategy rather than replacing it.

Finally, the design separates the decision of where to apply the optimization from the low-level mechanism that implements it. The image-level component decides which methods and send sites should be considered, and which receiver classes should be passed to the VM. The VM-level component is responsible for validating that information, generating the corresponding native-code dispatch structure, and patching the selected send site.

3.2. Trade-offs

RQ4 focuses on the VM-level cost of proactive cache generation. The main trade-off introduced by the proposed approach is between earlier dispatch optimization and additional VM-level work. Proactive generation can reduce the initial cost of dynamic dispatch, but it does so by generating dispatch structures before the VM would normally construct them reactively.

This trade-off is visible in the VM statistics. The PIC scenario generally increases the number of generated code structures and the amount of code space occupied by JIT-compiled methods and closed PICs. This additional generated code is the cost paid to reduce the initial dispatch overhead. Therefore, proactive generation should not be treated as a cost-free optimization.

The trade-off also depends on execution length. When a benchmark is executed only a small number of times, the initial cost of reactive cache construction is more visible. In that setting, installing the cache proactively can provide a larger relative benefit. As the number of iterations increases, the current baseline has more opportunities to build and reuse its caches reactively, and the initial cache-construction cost becomes amortized over a longer execution.

For longer executions, other costs may become more relevant than the initial dispatch cost. This is particularly visible in benchmarks such as `copy` and `asArray`, where the measured execution time is also affected by object allocation and heap usage. In those cases, the relative benefit of proactive cache generation can become smaller because the benchmark is no longer dominated only by dispatch-cache construction.

Overall, the results suggest that proactive IC/PIC generation is most useful when the cost of reaching an optimized dispatch state is relevant with respect to the total execution time. When that cost is quickly amortized, or when other sources of cost dominate the benchmark, the relative benefit becomes smaller.

3.3. Limitations

The current implementation makes several design decisions that define the scope of the results. First, proactive cache generation is currently applied only to send sites whose receiver is an instance variable. This decision keeps the image-level analysis focused and provides a direct path from the type information available in the image to the selected send sites. It should not be interpreted as a fundamental limitation of the approach: the same VM-level mechanism could be driven by other sources of receiver type information.

Second, the implementation generates monomorphic inline caches and closed PICs with a bounded number of entries. In the current version, closed PICs are generated for up to six receiver classes. This bound was chosen to focus the evaluation on monomorphic, bimorphic, and low-polymorphic sends, which are the cases most naturally represented by compact closed PICs. Send sites with a larger number of receiver classes are therefore outside the scope of the current evaluation.

Third, the generated cache is based on the receiver classes selected by the image-level analysis. In the current implementation, these classes come from the types observed for the instance variable used as the receiver. Therefore, the analysis does not distinguish different execution contexts in which the same variable may be used. This keeps the implementation simple, but may generate cache entries that are not used by every execution of a particular send site.

Finally, the evaluation is based on controlled synthetic benchmarks. This setup makes it possible to vary the number of send sites, the number of PIC entries, and the number of iterations independently. However, it does not capture all the interactions that may appear in larger applications, where program behavior, object allocation, method changes, and cache invalidation patterns can be more complex.

References

- [1] Cuis Smalltalk, <https://cuis.st/>, nd.
- [2] Cuis University, <https://sites.google.com/view/cuis-university>, nd.